

✓CORE JAVA YOU MUST PERFECT *BEFORE* DSA (MINIMUM + SUFFICIENT)

This is the **exact checklist**. If any item here is weak, DSA will feel painful.

☐ 1. Java Language Basics (NON-NEGOTIABLE)

You must be **fluent**, not just aware.

- Data types
- Variables
- Operators
- `if / else`
- `switch`
- `for, while, do-while`
- `break, continue`

☐ You should write loops **without thinking**.

☐ 2. Methods & Parameter Passing

You must clearly know:

- Method definition & call
- Return values
- Pass-by-value (very important)
- Method overloading

☐ DSA = calling helper methods correctly.

☐ 3. Arrays (EXTREMELY IMPORTANT)

You must master:

- 1D arrays
- 2D arrays
- Traversal

- In-place modification
- Passing arrays to methods

☐ **Most DSA problems start with arrays.**

☐ **4. Strings & StringBuilder**

You must know:

- `String` immutability
- Common string methods
- `StringBuilder` vs `String`
- Character arrays

☐ Many LeetCode problems are string-based.

☐ **5. Basic OOP (ONLY WHAT DSA NEEDS)**

You do **NOT** need full OOP theory.

You only need:

- Class & object
- Constructors
- `this` keyword
- Encapsulation (basic idea)

✗ You do NOT need inheritance, interfaces, polymorphism yet.

☐ **6. Recursion (BASIC LEVEL)**

You must understand:

- Base case
- Recursive call
- Stack frame idea

Example:

```
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

☐ Trees, DFS, DP all rely on this.

☐ 7. Java Collections (MINIMUM SET)

You must be comfortable with:

- ArrayList
- HashMap
- HashSet
- Stack
- Queue (LinkedList)

You do **NOT** need:

- TreeMap
 - PriorityQueue (yet)
 - Concurrency collections
-

☐ 8. Basic Input / Output

You should know:

- Reading input
 - Printing output
 - Fast I/O basics (Scanner vs BufferedReader awareness)
-

✗CORE JAVA YOU SHOULD *NOT* WAIT FOR

Delay these until **after DSA**:

- Multithreading
- File handling

- JDBC
- Swing / AWT
- Applets
- Reflection
- Annotations

They don't help with DSA.

☐ **READY-FOR-DSA TEST (VERY IMPORTANT)**

You are **ready to start DSA** if you can:

- ✓ Write a loop-based solution without IDE help
- ✓ Manipulate arrays in place
- ✓ Use HashMap for frequency counting
- ✓ Write simple recursive code
- ✓ Read & understand Java code quickly

If any of these feel shaky → fix them **this week**.

☐ **Correct transition plan**

Core Java (DSA-ready subset)

↓

DSA in Java

↓

LeetCode + practice

Not:

Complete Core Java book

↓

Start DSA

That path wastes time.

Final truth (memorize this)

Core Java is a tool.

DSA is the skill.

Learn the tool just enough to build the skill.
